

Program 6: Deploy an Automation build pipeline using Dockerhub

Step 1: mkdir program6
cd program6

Step 2: Create a Dockerfile
nano Dockerfile

Step 3: Create a PAT token
PAT for Dockerhub
PAT for Github
Copy the passwords and save it in a file

Step 4: initialize the npm packages

```
(base) 1rv24mc043_Honey@Yoga-7-2-in-1-14IML9:~/program6$ npm init -y
Wrote to /home/honey/program6/package.json:
```

```
{
  "name": "program6",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

```
server.js
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Hello from Docker automated pipeline!");
});

app.listen(3000, () => console.log("Server running on port 3000"));
```

Step 5: Login to the dockerhub using the PAT

 **Info** → A Personal Access Token (PAT) can be used instead.
To create a PAT, visit <https://app.docker.com/settings>

Password:

Configure a credential helper to remove this warning. See <https://docs.docker.com/go/credential-store/>

Login Succeeded

Step 6: Build the image

```
[+] Building 315.7s (11/11) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                 0.0s
=> => transferring dockerfile: 285B                                0.0s
=> [internal] load metadata for docker.io/library/node:18          2.8s
=> [auth] library/node:pull token for registry-1.docker.io         0.0s
=> [internal] load .dockerignore                                    0.0s
=> => transferring context: 2B                                       0.0s
=> [1/5] FROM docker.io/library/node:18@sha256:c6ae79e38498325db67193d 311.5s
=> => resolve docker.io/library/node:18@sha256:c6ae79e38498325db67193d39 0.0s
=> => sha256:c6ae79e38498325db67193d391e6ec1d224d96c693a 6.41kB / 6.41kB 0.0s
=> => sha256:eb29363371ee2859fad6a3c5af88d4abc6ff7d399ad 2.49kB / 2.49kB 0.0s
=> => sha256:37927ed901b1b2608b72796c6881bf645480268e 24.02MB / 24.02MB 41.8s
=> => sha256:79b2f47ad4443652b9b5cc81a95ede249fd97631 64.40MB / 64.40MB 55.4s
```

Step 7: Tag the image for dockerhub

Step 8: Push the image to the dockerhub

```
97c0489dc73d: Pushed
d9e96791b291: Pushed
879d814895cf: Pushed
736b2d292483: Pushed
d2a991bcab4d: Mounted from library/node
b624aa2d5ea2: Mounted from library/node
d399c9dc306f: Mounted from library/node
84f9fa179c1b: Mounted from library/node
ce84ba212e49: Mounted from library/node
e4dc8cd9ecc8: Mounted from library/node
6428cc293366: Mounted from library/node
2f7436e79a0b: Mounted from library/node
latest: digest: sha256:3332f9c7b0733f2a0f769f2cd04f8400b1d11037080dc3c5dc7da590321b0ef7 size: 2831
```

Step 9: Login to the ghcr using the PAT

Step 10: Tag the image for the GHCR

Step 11: Push the image to the GHCR

```
97c0489dc73d: Pushed
d9e96791b291: Pushed
879d814895cf: Pushed
736b2d292483: Pushed
d2a991bcab4d: Pushed
b624aa2d5ea2: Pushed
d399c9dc306f: Pushed
84f9fa179c1b: Pushed
ce84ba212e49: Pushed
e4dc8cd9ecc8: Pushed
6428cc293366: Pushed
2f7436e79a0b: Pushed
latest: digest: sha256:60456f382057407af2a054df8bdfc7ac8b93eb144480a95e2038549031423c65 size: 2831
```

Step 12: Pushing the files to the Github repo

Step 13: Check the following for the pushed files



